

# Stage 3 Step-by-Step Practical & Conceptual Recap

## 1. Middleware Architecture Setup (express.json())

- **What You Did:** In `server.js`, you registered `app.use(express.json());` at the root level of your application pipeline before any route declarations.
- **Sysadmin Analogy:** This acts like **Deep Packet Inspection (DPI)** or an **Ingress API Gateway**. By default, incoming TCP HTTP payloads arrive at the server port as raw unparsed byte streams. `express.json()` inspects the incoming `Content-Type: application/json` header, unmarshals the raw JSON byte payload, and populates `req.body` so route handlers can read it natively.

## 2. Daemon Process Control

- **What You Executed:** In your **left VS Code terminal tab**, you interrupted the running process with `Ctrl + C` and restarted the server daemon:  
`node server.js`
- **Observed Daemon Output:**  
Task API Daemon running on `http://localhost:3000`
- **Sysadmin Analogy:** Re-executing `node server.js` reloads updated script instructions into active system RAM, similar to executing `systemctl restart myapp` after editing configuration files in `/etc/`.

## 3. Initial POST Execution & PowerShell Syntax Diagnosis

- **What You Executed First:** In your **right VS Code terminal tab**, you ran an initial `curl.exe` POST test:  
`curl.exe -i -X POST http://localhost:3000/tasks -H "Content-Type: application/json" -d '{"title":"Buy milk"}'`
- **The Issue Encountered:** PowerShell stripped the escaped inner double quotes, causing `curl.exe` to transmit malformed JSON (`{title:Buy milk}`) to the daemon.
- **Observed Error Output:**  
`HTTP/1.1 400 Bad Request`  
`SyntaxError: Expected property name or '}' in JSON at position 1`
- **Sysadmin / Firewall Analogy:** Your Express server acted as an automated **Web Application Firewall (WAF)**. When malformed JSON passed through `express.json()`, the

parser dropped the invalid packet at the edge and returned 400 Bad Request rather than letting corrupt data enter system memory.

## 4. Successful Payload Transmission (HTTP 201 Created)

- **The Resolution Command:** You enclosed the JSON payload in single quotes `'{"title":"Buy milk"}'` to prevent PowerShell string interpolation:  
`curl.exe -i -X POST http://localhost:3000/tasks -H "Content-Type: application/json" -d '{"title":"Buy milk"}'`
- **Observed Response:**  
HTTP/1.1 201 Created  
Content-Type: application/json; charset=utf-8  
Content-Length: 40  
  
`{"id":4,"title":"Buy milk","done":false}`
- **Sysadmin Analogy:** HTTP 201 Created serves as an official server receipt confirming a new persistent resource was instantiated in system state, complete with a dynamic primary key (ID 4) assigned via dynamic memory allocation (like a DHCP server issuing an IP lease).

## 5. Verification of State Persistence (GET /tasks)

- **What You Executed:**  
`curl.exe -i http://localhost:3000/tasks`
- **Observed Response Body:**  

```
[  
  {"id":1,"title":"Responding to routine client emails","done":false},  
  {"id":2,"title":"Monitoring server status alerts","done":false},  
  {"id":3,"title":"Prepare for client meeting","done":false},  
  {"id":4,"title":"Buy milk","done":false}  
]
```
- **Verification Outcome:** Confirmed that the new record id: 4 was successfully pushed into the active array cache in RAM.

## 6. Input Validation Firewall Test (HTTP 400 Bad Request)

- **What You Executed:** Sent an empty JSON payload `{}` to test defensive validation rules:

```
curl.exe -i -X POST http://localhost:3000/tasks -H "Content-Type: application/json" -d '{}'
```

- **Observed Response:**

HTTP/1.1 400 Bad Request

```
{"error":"Title is required and must be a non-empty string"}
```

- **Sysadmin Analogy:** Standard defensive security practice dictates that system endpoints **never trust incoming client data**. Rejecting blank payloads with HTTP 400 prevents corrupt, empty, or malicious records from populating downstream system memory or databases.

## 7. Saving Disk State & Git Restore Point

- **What You Executed:** Saved server.js (Ctrl + S), then staged and committed all changes in the right terminal:  
git add .  
git commit -m "stage 3: create with validation"
- **Observed Audit Output:**  
[master 70062a5] stage 3: create with validation
- **Sysadmin Analogy:** Executing git commit writes an immutable system snapshot (70062a5), establishing a clean restore point before implementing record updates (PUT/PATCH) and deletions (DELETE) in Stage 4.